

演講稿 | 多模態信用風險智能評估架構

Joey Wu · 2026 年 5 月

共 15 張投影片 | 預計 9 分鐘

Slide 1 | 開場 (45 秒)

讓我先說一個很多人沒有意識到的事。

一個多元金融集團，同時經營壽險、產險、銀行、證券業務，手上擁有的客戶資料廣度，遠超任何一家純銀行。保單繳費紀錄、理賠溝通記錄、投資往來歷史——這些資料，天然就是多模態的。

但現在，它們都在各自的系統裡，從來沒有被整合進信用決策。

這個系統要回答的問題，就是：如果我們把這些被浪費的資料整合起來，讓每一個信貸決策同時看到這個客戶的完整輪廓——會長什麼樣子？而且，每一個決策都能被解釋、被追責。

Slide 2 | 為什麼單純的表格模型不夠 (45 秒)

讓我把問題說具體。

同樣逾期兩次——三年前的舊帳，和最近三個月連續惡化，在表格模型眼裡是同一個數字。軌跡，是看不到的。

再想像一個借款人，個人財務數字還好，但他的擔保人背後有一個共同擔保的圈子，圈子裡已有三個人陸續違約，風險正沿線蔓延。這個傳染路徑，表格模型完全看不到。

最後，銀行每天收申請書，裡面有借款用途、還款計畫——而我們這套新技術，能把這些被浪費的資訊，轉化為精準的風險控制判斷。

三個盲點，我們同時解決。

Slide 3 | 四種訊號，一個借款人的完整側寫 (30 秒)

這四種資料是正交的，各自回答不同問題。

用一個比喻：表格是病歷表，時序是複診紀錄、可以看到趨勢，圖是候診室裡跟誰坐在一起，文字是走進診間說的第一句話。

真實銀行本來就擁有這四種資料。這個系統回答的問題只有一個：把它們全部用上的架構，長什麼樣子？

Slide 4 | 系統架構——四個 Encoder，一個決策（40 秒）

從左讀到右。四種資料各自進入最適合的 Encoder——XGBoost 處理表格，Bi-LSTM 處理時序，GraphSAGE 處理關係圖，sentence-BERT 處理申請文字。每個 Encoder 輸出一個 32 維向量，格式統一才能拼接。

四個 32 維向量合成 128 維，送進三層 MLP（神經網路分類器），輸出違約機率。接著分兩條路：SHAP 解釋加 GenAI 報告，以及分析師 Dashboard。

最關鍵的設計性質：每個 Encoder 只承諾輸出 32 維，任何一個換掉，下游完全不需要改。

Slide 5 | 表格 + 時序：XGBoost 與 Bi-LSTM（50 秒）

表格用 XGBoost，時序用雙向 LSTM——兩個模態都已升級至真實資料。

XGBoost 的技巧叫葉節點 Embedding——每棵決策樹把借款人分到一個葉節點，同一葉節點代表信用行為等價，把這個分群知識轉成 32 維向量帶進融合層。

但更重要的是調優過程。原版用 1,200 筆失真樣本，AUC 只有 0.67。換成完整 15 萬筆資料，加上 Optuna 跑 100 次超參數搜尋、Stratified 5-Fold 交叉驗證、再加入 LightGBM 和 CatBoost 做三模型 Ensemble——最終 OOF AUC 達到 **0.866**。這個數字用五折交叉驗證確認，不是單次 split 的運氣。

LSTM 的資料來源是 Lending Club 10 萬筆真實貸款。用前 10 個月預測第 11 到 12 個月是否逾期，嚴格的時間切割確保沒有 data leakage。序列相關性經過統計驗證：90% 的違約者，在違約發生前就已出現逾期前兆。違約不是突然的，它有連環效應。

Slide 6 | 圖 + 文字：GraphSAGE 與 sentence-BERT（35 秒）

圖模型捕捉的是傳統信用評分看不到的東西——一個擔保人圈子裡，一個人違約，第二個、第三個跟上。GraphSAGE 兩層讓每個節點看到「鄰居的鄰居」，群聚風險的傳染路徑就在這裡。新借款人不需要重訓整個圖，直接套用已學好的函數。

文字用 sentence-BERT，讀語境不只讀字。同樣一句「我需要一筆錢來償還債務」——出現在完整申請書裡，和出現在一封語氣急迫的求助訊息裡，傳遞的風險信號截然不同。傳統關鍵字系統兩個給一樣的分數，sentence-BERT 不會。

Slide 7 | Late Fusion——為什麼在決策層才合併（35 秒）

Late Fusion 的核心邏輯：讓每個模型做它最擅長的事，只在最後才合在一起。

早期融合會毀掉各自的歸納偏置——決策樹靠特徵分裂，LSTM 靠開門，GNN 靠鄰域聚合，Transformer 靠 Attention，放在同一個輸入層就全部消失。

兩個實際好處：資料缺失時補零向量，系統不崩潰；各模型程式碼完全分開，LSTM 出問題不影響 XGBoost，容易獨立 Debug 和維護。

Slide 8 | 實驗結果（50 秒）

這張表有三條故事，每條都說明一件事。

AUC 是排名能力——隨機抽一個違約者和一個正常人，模型有多少機率把違約者排前面。

第一條：表格模態。 原本 1,200 筆失真樣本是 0.67。換成完整 15 萬筆，Optuna 調優 XGBoost、加 LightGBM 和 CatBoost、三模型 Ensemble，最終 OOF AUC **0.866**，用 Stratified 5-Fold 驗證，數字可信。

第二條：時序模態。 Bernoulli 合成資料是 0.72。換成 Lending Club 真實資料後跳到 0.97——90% 的違約者在違約前就已出現逾期前兆，LSTM 學到了這個連環效應。

第三條，也是最重要的：Fusion。 原版 Fusion 只有 0.74——但問題不是架構，是資料污染，四個模態來自不同的借款人群體。把四個模態全部對齊到同一批 15 萬筆借款人之後，Fusion AUC 從 0.74 跳到 **0.864**，跟純表格 Ensemble 的 0.866 幾乎相同。這證明了一件事：Late Fusion 架構是對的，差距來自資料，不是設計。

Slide 9 | 可解釋性——三層鏈路，每一層都可審計（35 秒）

這套系統的解釋分三層。

第一層，特徵層：哪個欄位把風險分數推高了多少。看這張圖——紅點集中在右側，代表信用使用率高的人，風險分數被系統性推高，跟信貸員直覺一致。

第二層，模態層：四種資料裡，是哪一種主導了這次判定。

第三層，GenAI 報告：把前兩層的數字，交給 AI 寫成信貸員讀得懂的中文報告。但規定它只能用已計算好的數字說話，不能自行推測——這樣 AI 的輸出才能進合規文件。

Slide 10 | 合規設計——可解釋，也可追責（30 秒）

三件事。

第一，拒貸理由。SHAP 直接輸出「主因：90 天逾期次數，風險上升 0.12 分」——有憑有據，符合金融消費者保護法要求。

第二，公平性。年齡、家庭人數可能偏向特定族群，需定期分群確認核准率是否一致。

第三，追責紀錄。每筆決策記錄模型版本、SHAP 前三名、人工覆蓋原因。事後查閱，30 秒重現完整決策脈絡。

Slide 11 | 限制與下一步（15 秒）

圖的邊不是真實擔保關係，文字是模板生成的一一這兩個模態換上真實資料，Fusion 才有機會超越純表格的 0.866。

架構不用改，資料是唯一的變數。

Slide 12 | 全球領先機構都在走這條路（25 秒）

這些不是新技術——XGBoost 信用評分是美國各大銀行的業界標準，Upstart 用 ML 分析申請書文字已與多家銀行合作，PayPal 和 Mastercard 用圖神經網路偵測詐欺網絡。

這個系統做的，是把這些已被驗證的元件整合在一起。台灣監管規則正在成形，現在進場的機構，會成為標準的制定者。

Slide 13 | Live Demo — Page 1：授信風險總覽（30 秒）

今天 180 筆評估，高風險占 30.6%，預估潛在損失 \$3M。

右邊這張圖是主管真正需要的東西：85 件複審、55 件拒絕、40 件核准——三個數字，今天的授信資源就這樣分配出去了。

下面 Top 10 清單，每一行都有分數、風險等級、主要成因、建議行動，可以直接交給信貸員執行。

Slide 14 | Live Demo — Page 2：模型可解釋性分析（35 秒）

這一頁回答一個問題：模型的判斷，可以被信任嗎？

左邊 SHAP 長條圖——循環信用使用率是最大驅動因子，跟信貸員直覺一致。

右邊兩張散點圖是交叉驗證：收入越低，高風險越集中；逾期越多，高風險越集中。模型的行為，跟一個資深信貸員的判斷是對齊的——這才是可以被信任的模型。

Slide 15 | Live Demo — Page 3：客群分群與授信策略（25 秒）

兩個發現。

資深年齡層低風險最多，但同樣有 14 人高風險——年齡不是保護因素，不能單獨判斷。

高風險借款人分散在所有負債比率區間，不只集中在高負債的地方——這正是為什麼單一指標不夠，需要四種訊號一起看。

Slide 16 | Live Demo — Page 4：借款人個案分析（收尾，45 秒）

最後一頁，信貸員的個案工具，也是整個系統最具體的展示。

下拉選擇任何一個借款人，分數、SHAP 值、建議行動即時更新。

以借款人 0 為例：評分 32%，中風險。SHAP 影響值 -0.14，是負的——代表循環信用使用率對他來說是壓低風險的力量，不是推高。32% 是其他特徵造成的。這個細節，傳統報表完全看不到。

最下方是 AI 生成授信摘要：「風險評分 32%，主要風險因子：循環信用使用率，建議人工複審。」這一句話，可以直接進入合規文件的拒貸理由欄位。

謝謝大家，歡迎提問。

時間總覽

Slide	主題	預估時間
1	開場	45 秒
2	為什麼不夠	45 秒
3	四種訊號	30 秒
4	系統架構	40 秒

Slide	主題	預估時間
5	XGBoost + Bi-LSTM	50 秒
6	GraphSAGE + sentence-BERT	55 秒
7	Late Fusion	35 秒
8	實驗結果	50 秒
9	可解釋性	45 秒
10	合規設計	40 秒
11	限制與下一步	40 秒
12	全球機構都在走這條路	30 秒
13	Demo Page 1	35 秒
14	Demo Page 2	40 秒
15	Demo Page 3	45 秒
16	Demo Page 4（收尾）	45 秒
合計		~9 分 40 秒

附錄 | 給非資訊背景主管的白話解釋

這段不是演講稿，是你向主管說明技術時的參考用語。

Bi-LSTM（雙向長短期記憶網路）

白話：一個能讀懂時間順序的 AI。

想像你在看一個借款人過去 12 個月的還款紀錄。一般人從頭讀到尾——「1 月正常、2 月正常、3 月開始逾期、4 月繼續逾期」。Bi-LSTM 的特別之處是它同時從頭往尾讀、也從尾往頭讀，把前後的脈絡都考慮進去，才做出判斷。

均值池化 (Mean Pooling)

白話：把所有月份的成績加起來取平均。

舉例：學生 12 個月的月考成績，前 10 個月都 90 分，最後 2 個月突然 30 分。均值池化算出來的平均是 81 分，看起來還不錯——但掩蓋了最近惡化的訊號。

我們選擇不用均值池化，改用「最後的隱藏狀態」——也就是 LSTM 綜合考慮過所有月份之後，在最後給出的一個綜合判斷。這個判斷更能反映趨勢，特別是「最近變差了」這種時序信號。

GraphSAGE (圖神經網路)

白話：一個能看懂「社會關係」的 AI。

傳統信用評分只看這個人本人——他的負債、收入、逾期紀錄。但銀行業有一個現象：詐欺和違約很少是孤立的，往往是一群人一起出問題，例如同一個仲介介紹的借款人、同一個擔保人簽保的一圈人。

GraphSAGE 的做法：把所有借款人畫成一張「關係網絡圖」，把財務特徵相似的人連線。然後對每一個借款人，不只看他自己，也看「他的鄰居在做什麼」、「他鄰居的鄰居在做什麼」。如果一個借款人周圍的人開始相繼違約，系統會把這個風險信號傳給他。

「SAGE」是演算法名稱，關鍵優勢是：新借款人申請進來，不需要把整個網絡重新算一遍，可以直接套用已學好的規則——這對每天都有新申請的銀行很重要。

sentence-BERT (句子理解模型)

白話：一個能讀懂申請書、理解意思的 AI，不只是查關鍵字。

傳統系統做文字分析，是找關鍵字——「這份申請書有沒有出現『償還』這個詞？」。但這太粗糙，同樣一句「我需要這筆錢」，放在一份條理清晰的申請書裡，和放在一封語氣慌張的求助信裡，傳遞的信號完全不同。

sentence-BERT 讀的是「整句話的意思」，能感知語境。它被用了大約 10 億個句子對訓練，學到了很深厚的語言理解能力。我們的做法是把它的語言理解能力「借來用」，再加一個小小的轉換層，讓它學會哪種申請書的說法跟違約風險有關。

Late Fusion（決策層融合）

白話：四個專家各自給意見，最後才開會投票，不是一開始就把所有資料混在一起。

Early Fusion（早期融合）的做法：把所有資料丟進同一個大鍋，煮成一個東西，再讓 AI 去分析。問題是不同類型的資料混在一起，反而互相干擾。

Late Fusion 的做法：財務數據給財務分析師（**XGBoost**），還款時序給行為分析師（**LSTM**），關係圖給社群分析師（**GraphSAGE**），申請書給文件審查員（**sentence-BERT**）。四個專家各自做完報告，最後召集在一起，綜合四份報告做最終判定。

這樣的好處：每個專家不會被其他人干擾，而且如果某份資料缺失（例如某個借款人沒有歷史還款紀錄），那個專家報告留白，其他三個人繼續正常工作，整個系統不會崩潰。

實驗結果（AUC）

白話：AUC 是模型「把壞人排在好人前面」的能力，不是答對率。

假設你有 100 位借款人，其中 10 位未來會違約。模型幫你把這 100 人按「違約可能性」由高到低排序。AUC = 0.8 代表：隨機抽一個真的違約者和一個正常人，模型有 80% 的機率把違約者排得比較前面。

這個系統的 AUC：

- **XGBoost 的 0.67：**用的是真實資料，但樣本太少，分佈也不對，所以偏低。完整資料的業界水準約 0.85。
- **LSTM 的 0.72 和 GraphSAGE 的 0.74：**這兩個用的是我自己產生的模擬資料，就像「用自己出的考題考自己」——分數看起來不錯，但不代表在真實資料上也一樣好。

一句話：這個系統驗證了架構設計是可行的，實際預測能力需要接上真實資料才能知道。